NEXORA

AI-Powered Student Productivity & Career Intelligence Platform

COMPLETE IMPLEMENTATION ROADMAP

Architecture → Phases → Pipelines → Development Gates → Testing → Deployment

Final Technology Stack

Frontend: React + Vite + Tailwind CSS + Recharts
Backend: Node.js + Express.js + JavaScript
Database: MongoDB + Mongoose
Authentication: JWT + secure refresh-token sessions
AI: Hosted pre-trained LLM behind a provider-agnostic adapter
Memory: MongoDB first; semantic/vector retrieval in Phase 1.5
Testing: Jest + Supertest + React Testing Library + Playwright
CI/CD: GitHub Actions
Deployment: Vercel + Render/Railway + MongoDB Atlas

This roadmap is derived from the uploaded NEXORA Final Architecture & Implementation Notes v4 and expands its 8-week roadmap into concrete engineering phases and checkpoints.

1. Project Execution Strategy

The architecture is frozen around a modular monolith: React handles the user experience, Node.js + Express handles APIs and business logic, MongoDB stores durable student data, and the AI engine builds bounded context before calling a hosted LLM. React never talks directly to MongoDB or the LLM provider.

The core development rule is vertical slices: complete a feature end-to-end across UI → API → service → database → test before moving to the next feature. The original architecture also recommends building the non-AI product before the AI system and cutting vector search/SSE first if the schedule slips.

Master Phase Order

Phase	Focus	Primary outcome
0	Product & architecture freeze	Scope, user journeys, repository and engineering rules
1	Development foundation	React ↔ Express ↔ MongoDB baseline + CI
2	Authentication & identity	Secure registration, login, refresh, protected routes
3	Core student platform	Tasks, study, profile, dashboard
4	Career + projects + analytics	Skills, career goals, projects, measurable progress
5	AI foundation	Provider adapter + working chat + conversation storage
6	NEXORA memory	Short/session/long-term memory + privacy controls
7	AI intelligence	Skill gap, roadmap, recommendations, feedback
8	Semantic memory + streaming	Vector retrieval + SSE, only after core is stable
9	Security + testing	Hardening, automated tests, E2E
10	CI/CD + deployment	Production-like deployed system
11	UI/UX polish	Responsive, accessible, polished experience
12	Final demo + documentation	Resume-ready, viva-ready evidence

2. Complete NEXORA System Pipeline

This is the main runtime architecture. The uploaded architecture defines the same overall flow: React → REST/SSE → Node.js + Express → API layer → services → MongoDB / AI engine → context builder → memory retriever → prompt manager → LLM adapter → personalized response.

Main Application Pipeline

```
USER
  ↓
React + Vite + Tailwind + Recharts
  ↓
REST / JSON        (+ SSE later for streaming chat)
  ↓
Node.js + Express
  ↓
/api/v1
  ↓
Middleware
  ■■■ Authentication
  ■■■ Validation
  ■■■ Rate limiting
  ■■■ Error handling
  ■■■ Logging
  ↓
Controllers
  ↓
Services / Business Logic
  ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  ↓              ↓                ↓
MongoDB        Analytics          AI Engine
  ↓                                 ↓
Student Data                        Context Builder
                                    ↓
                                Memory Retriever
                                (keyword → vector)
                                    ↓
                                 Prompt Manager
                                    ↓
                             LLM Provider Adapter
                                    ↓
                              Hosted Pre-trained LLM
                                    ↓
                                Response Parser
                                    ↓
                                JSON / SSE response
                                    ↓
                                    React
                                    ↓
                                Student action
                                    ↓
                                    MongoDB
```

Key rule: React never accesses MongoDB directly and never exposes the LLM/provider secret. All private operations pass through the backend.

Non-AI Request Pipeline

React → API service → route → controller → service → Mongoose model → MongoDB → JSON response → React

AI Request Pipeline

React → /api/v1/ai/chat → controller → conversation/memory services → memory retriever → context builder → prompt manager → LLM client → provider → response parser → JSON/SSE → React → save messages → optionally save memory.

3. PHASE 0 — Product & Architecture Freeze

Goal: remove ambiguity before implementation.

Tasks

- Freeze the final stack: React/Vite, Tailwind, Recharts, Node/Express, MongoDB/Mongoose, JWT, hosted LLM API, provider adapter.
- Write the product statement and define the target student workflow.
- Define core screens: Login, Register, Dashboard, Tasks, Study, Career, Projects, Analytics, AI Chat, Settings.
- Define user journeys: onboarding, task tracking, study tracking, career planning, AI coaching and memory controls.
- Freeze scope: no LLM-from-scratch training, no microservices, no Kubernetes, no Kafka, no multiple databases.
- Create GitHub repository, branch strategy, README skeleton and CHANGELOG.

Deliverable / Gate

Approved feature list + architecture diagram + user journey + repository + development conventions.

4. PHASE 1 — Development Foundation

Goal: establish the application skeleton and prove the base stack works end-to-end.

Backend

- Initialize Node.js + Express project.
- Create server.js and app.js.
- Create /api/v1 versioned API structure.
- Add environment configuration, MongoDB connection and global error handling.
- Create GET /api/v1/health.

Frontend

- Initialize React + Vite.
- Add Tailwind CSS and routing.
- Create base layout, navigation, common Button/Input/Modal/Loader/Error components.
- Create API service layer so pages do not contain raw HTTP logic.

Engineering

- Set up Git branches and pull-request workflow.
- Add lint/test commands.
- Create initial GitHub Actions workflow.

Gate 1

React ↔ Express ↔ MongoDB works. Health endpoint works. CI is green.

5. PHASE 2 — Authentication & User Identity

Goal: make NEXORA a secure multi-user application before adding private student data.

Implementation

- Create User model with profile, preferences and careerGoal fields.
- POST /api/v1/auth/register.
- POST /api/v1/auth/login.
- Short-lived JWT access token.
- Refresh-token rotation with httpOnly + Secure cookie.
- Store only hashed refresh tokens in authSessions.
- Implement logout, revocation and refresh-token family reuse detection.
- Create authentication middleware for protected routes.
- GET /api/v1/auth/me.
- GET/PUT /api/v1/users/me.
- DELETE /api/v1/users/me with user-owned data cleanup.

Security baseline

- Argon2id preferred; bcrypt acceptable.
- Never expose provider/API secrets to React.
- Every protected database query must be scoped to authenticated userId.
- Validate and sanitize incoming data.

Gate 2

Register → login → protected dashboard/API → refresh → logout → account deletion works.

6. PHASE 3 — Core Student Platform

Goal: build a complete non-AI student productivity product first.

Tasks module

- Task model: userId, title, status, priority, deadline, timestamps.
- GET/POST /api/v1/tasks.
- PUT/DELETE /api/v1/tasks/:id.
- Completion, status, priority and deadline handling.
- Pagination for list endpoints.

Study module

- StudySession model: userId, subject, duration, date, notes.
- GET/POST /api/v1/study/sessions.
- Track study history and totals.

Dashboard

- Profile summary.
- Task completion rate.
- Study hours.
- Current goals.
- Initial Recharts widgets.

Important rule

Normal analytics calculations stay in backend JavaScript. Do not ask the LLM to calculate basic numbers.

Gate 3

A student can register, manage tasks, record study sessions and see meaningful dashboard metrics without any AI.

7. PHASE 4 — Career + Projects + Analytics

Goal: create the structured data layer that later makes NEXORA's AI personalized.

Career intelligence data

- Create Skill and CareerGoal models.
- Seed careers, skills, roadmaps and project templates.
- Allow the student to choose a target role and maintain current skill levels.

Deterministic skill-gap engine

```
Required skill level
        -
Student skill level
        =
Skill gap

Example:
React required = 80
React current  = 40
Gap            = 40
```

Projects

- Project model with userId, title, techStack and status.
- GET/POST /api/v1/projects.
- Connect projects to career progress where useful.

Analytics

- GET /api/v1/analytics/dashboard.
- GET /api/v1/analytics/productivity.
- Show task, study, skill and career-readiness trends.

Gate 4

NEXORA is a useful non-AI student productivity + career platform with structured data ready for AI context.

8. PHASE 5 — AI Foundation

Goal: integrate a hosted pretrained LLM without coupling the application to one provider.

AI architecture

```
ai/
■■■ providers/
■   ■■■ provider-A.adapter.js
■   ■■■ provider-B.adapter.js   (optional)
■■■ llmClient.js
■■■ contextBuilder.js
■■■ promptManager.js
■■■ memoryRetriever.js
■■■ embeddings.js               (Phase 1.5)
■■■ responseParser.js
```

Build order

1. Create provider interface/adapter contract.
2. Implement llmClient.js.
3. Create POST /api/v1/ai/chat.
4. Build AI service/controller flow.
5. Create chat UI.
6. Create Conversation and Message models.
7. Store user and assistant messages.
8. Initially use bounded recent conversation context only; no semantic memory yet.

Gate 5

Student can open a conversation, send a message, receive a hosted-LLM response and see saved conversation history.

9. PHASE 6 — NEXORA Memory System

Goal: turn generic chat into persistent, user-controlled continuity.

Three memory levels

Memory	Meaning	Example
Short-term	Current conversation context	Previous message needed to understand 'it'
Session	Facts relevant to current chat	Current topic: React Hooks
Long-term	Important facts saved across chats and retrieved by relevance	Career goal, preferences, rejected ideas

Implementation

- Create Memory model: userId, memory, type, importance, embedding?, timestamps.
- Implement memory.service.js.
- Implement memoryRetriever.js using keyword/category filtering for the MVP.
- Implement contextBuilder.js to combine profile, skills, activity, relevant memories, recent messages and current request.
- Keep V1 context bounded; the architecture recommends current request + relevant profile/context + last 15 active-conversation messages.

- Do not store every chat sentence as permanent memory.

Memory controls

- AI Memory ON/OFF.
- Remember profile ON/OFF.
- Remember conversations ON/OFF.
- Remember preferences ON/OFF.
- View saved memories.
- Delete one memory / delete all memories.
- Fresh/private chat that excludes long-term reads and writes.

Gate 6

NEXORA can use relevant saved context across chats, while a fresh/private chat can intentionally ignore long-term memory.

10. PHASE 7 — AI Intelligence & Recommendations

Goal: connect AI to NEXORA's structured student/career data so it becomes more than a generic chatbot.

AI Skill Analysis

Student profile + target career + current skills → deterministic skill-gap calculation → context builder → LLM explanation + recommended actions.

AI Career Roadmap

Target role + skill gaps + projects + progress → roadmap generation → save roadmap → track progress.

AI Recommendation

Career relevance + deadline urgency + skill gaps + progress + previous feedback → recommendation → feedback storage.

Feedback-aware loop

```
Recommendation
   ↓
Helpful / Not useful / Rejected / Accepted
   ↓
Store explicit feedback
   ↓
"Give another"
   ↓
Retrieve rejected/seen items
   ↓
Build "do not repeat" context
   ↓
LLM generates a meaningfully different recommendation
```

Key engineering principle

Your backend should calculate facts and metrics; the LLM should explain, reason over supplied context and generate personalized language/actions. This separation makes the system more reliable and easier to evaluate.

Gate 7

NEXORA can perform personalized skill analysis, career roadmap generation and feedback-aware recommendations using real student data.

11. PHASE 8 — Semantic Memory + SSE Streaming

Goal: add advanced retrieval and chat UX only after the core system is stable.

Semantic memory pipeline

```
Memory candidate
   ↓
Embedding generation
   ↓
MongoDB memory document + embedding
   ↓
Vector index
   ↓
Future user question
   ↓
Query embedding
   ↓
Vector similarity search
   ↓
Top relevant memories
   ↓
contextBuilder.js
   ↓
LLM
```

SSE streaming pipeline

```
User sends message
   ↓
Backend calls LLM
   ↓
Tokens arrive incrementally
   ↓
SSE stream
   ↓
React renders incrementally
```

Cut-line

If the schedule slips, cut vector search and SSE first. Never cut authentication, core security, tests or deployment.

Gate 8

Optional advanced milestone: semantic memory retrieval and/or streaming chat work without destabilizing the core application.

12. PHASE 9 — Security + Testing Hardening

Goal: prove the system is safe, deterministic and testable.

Security checklist

- Secrets in .env only; never commit credentials.
- Short-lived access tokens + refresh-token rotation/revocation.
- User ownership checks at the service layer.
- Input validation and sanitization.
- Separate general and AI rate limits.
- Prompt-injection hygiene: delimit system instructions, user input, retrieved memory and conversation text.
- Never treat stored memory as an instruction.

- Validate/sanitize AI output before rendering.
- Global error handling + structured logging.
- Account deletion cleans user-owned data according to the project data policy.

Testing strategy

Area	Tool	Minimum coverage
Backend	Jest + Supertest	Auth, tasks, career, memory, AI, recommendations
Frontend	React Testing Library + Jest	Forms, dashboard, chat, protected routes
AI pipeline	Mock provider	Prompt building, memory filters, response parsing, adapter contract
E2E	Playwright	Login → task → chat → AI response → memory/privacy → dashboard

CI rule

Never call the real remote LLM provider from CI tests. Mock the provider adapter so tests stay deterministic, fast and free of provider/API-cost dependencies.

13. PHASE 10 — CI/CD + Deployment

Goal: make NEXORA a reproducibly testable and deployable application.

CI pipeline

```
Pull Request
   ↓
Install dependencies
   ↓
Lint
   ↓
Backend tests
   ↓
Frontend tests
   ↓
PASS? ■■ NO → Block merge
   ■
  YES
   ↓
Merge to main
   ↓
Deploy backend + frontend
```

Deployment topology

```
Internet
  ■■■ Vercel
  ■     ■■■ React frontend
  ■
  ■■■ Render / Railway
        ■■■ Node + Express backend
                ■■■ MongoDB Atlas
                ■■■ Hosted LLM API
```

Production checklist

- Environment variables configured.
- MongoDB Atlas connected.
- Backend health endpoint deployed.
- Frontend points to deployed /api/v1 backend.
- CI passes.
- E2E smoke test passes against deployed/staging environment.
- No secrets in Git.

Gate 10

A real deployed NEXORA environment is accessible and the main user flow works.

14. PHASE 11 — UI/UX Polish

Goal: turn the working system into a polished product.

Dashboard polish

- Career readiness score.
- Productivity score.
- Study hours/streak.
- Skill progress.
- AI insight card.
- Today's priorities.

Product polish

- Responsive layout.
- Consistent typography, spacing and component design.
- Loading/skeleton states.
- Empty states.
- Error states.
- Accessible controls and keyboard-friendly flows.
- Clear memory/privacy controls.
- Polished AI chat with optional streaming.
- Useful charts without visual clutter.

Gate 11

A new user can understand the product quickly, complete the core flows without confusion, and the interface looks consistent across desktop/mobile layouts.

15. PHASE 12 — Final Demo + Documentation

Goal: package NEXORA as a resume-ready and viva-ready engineering project.

Documentation

- README with product description, screenshots, setup, architecture and deployment.
- docs/architecture.md.
- docs/api.md.
- docs/database.md.
- Architecture Decision Records where useful.
- CHANGELOG.md showing development progress.
- API documentation through OpenAPI/Swagger + Postman collection.

Final demo story

1. Register and complete the student profile.
2. Set a target career.

3. Add current skills and skill levels.
4. Create a few tasks and study sessions.
5. Open the career dashboard and show deterministic skill-gap analysis.
6. Generate a personalized roadmap.
7. Ask NEXORA AI Coach what to focus on today.
8. Show that the AI receives profile/career/productivity context.
9. Save a preference/rejected recommendation as memory.
10. Ask for another recommendation and demonstrate that the rejected idea is avoided.
11. Show analytics, memory controls and privacy/fresh-chat behavior.
12. Show deployed application, tests and CI status.

Gate 12

NEXORA is deployed, tested, documented and demonstrable from onboarding to personalized AI action.

16. 8-Week Master Schedule

Week	Primary focus	Expected result
1	Project setup, Git workflow, React shell, Express skeleton, MongoDB, JWT auth, CI	Register/login/protected route end-to-end; CI green
2	Tasks + Study + Dashboard shell	Core CRUD works without AI; first analytics widgets live
3	Career + Projects + Analytics	Non-AI product usable and demoable
4	Chat UI + conversation storage + LLM provider adapter	Working AI chat with no memory yet
5	Short/session/long-term memory + memory controls + context builder	Personalized AI chat across sessions
6	Embeddings/vector retrieval + SSE streaming	Optional semantic retrieval + live streaming; cut first if schedule
7	Feedback-aware recommendations + skill-gap + roadmap AI	AI features connected to career/skills data
8	Polish, accessibility, Playwright, docs, deployment, demo	E2E-verified, deployed, resume-ready project

Vertical-slice rule

```
DO NOT:
Frontend for 2 weeks → Backend for 2 weeks → AI later

DO:
Feature
  ↓
UI
  ↓
API
  ↓
Controller
  ↓
Service
  ↓
Model / MongoDB
  ↓
Test
  ↓
DONE
```

Example: Create Task

React Tasks page → task.service.js → POST /api/v1/tasks → route → controller → task.service.js → Task.model.js → MongoDB → JSON response → React.

Example: AI Chat

React AIChat.jsx → ai.service.js → POST /api/v1/ai/chat → controller → conversation/memory services → memoryRetriever → contextBuilder → promptManager → llmClient → provider → responseParser → JSON/SSE → React → save conversation + optional memory.

17. Two-Person Team Execution Plan

Area	Person 1 — primary	Person 2 — primary	Shared
Frontend	React, UI/UX, dashboard, tasks, study, career UI, shared UI	API integration, state	Design decisions
Backend	Support/API integration	Express, MongoDB, auth, services, architecture	Architecture
AI	AI UX + prompt testing	AI adapter, memory, context, recommendations	Behavior/eval
Testing	Frontend tests + E2E support	Backend/integration tests	E2E smoke flow
Deployment	Frontend deployment	Backend/database deployment	CI/CD + release
Docs	UI/product documentation	API/architecture/database docs	README, CHANGELOG, demo

18. Development Gates — Do Not Move Forward Too Early

Gate	Must be working
1	React ↔ Express ↔ MongoDB + CI
2	Registration, login, protected routes, refresh, logout, account deletion
3	Tasks + study + dashboard without AI
4	Career + skills + projects + analytics
5	AI provider adapter + working chat + conversation storage
6	Memory retrieval + privacy controls + context builder
7	Skill gap + roadmap + recommendations + feedback
8	Optional vector/SSE without destabilizing core
9	Security + unit/integration/frontend/E2E tests
10	Deployed frontend/backend/database + CI
11	Polished responsive and accessible UI
12	Final documentation + demo + resume/viva evidence

19. Final NEXORA Architecture Checklist

Layer	Final choice
Frontend	React + Vite + Tailwind + Recharts
Backend	Node.js + Express + JavaScript
Database	MongoDB + Mongoose
API	REST + /api/v1
Auth	JWT access + refresh-token rotation/revocation
AI	Hosted pre-trained LLM API
AI integration	Provider-agnostic adapter
Memory	MongoDB first; vector retrieval in Phase 1.5
Chat	Normal JSON first; SSE after stable chat
Testing	Jest + Supertest + React Testing Library + Playwright
Docs	OpenAPI/Swagger + Postman
CI/CD	GitHub Actions
Deployment	Vercel + Render/Railway + MongoDB Atlas
Monitoring	Sentry + structured logging as polish
AI cost control	Per-user budget, token cap, rate limits, circuit breaker, usage telemetry

20. Scope Cut-Line — What to Cut First

If the schedule slips, preserve the engineering fundamentals and remove advanced extras in this order:

1. Cut SSE streaming.
2. Cut semantic/vector memory.
3. Cut Redis or other performance infrastructure unless profiling proves it is needed.
4. Reduce visual animation/polish.
5. Reduce optional monitoring features.

Never cut

- Authentication and authorization.
- User data ownership/isolation.
- Security basics.
- Core student workflows.
- AI provider adapter.
- Core memory/privacy behavior.
- Automated tests and the Playwright smoke flow.
- Deployment.
- Documentation.

21. One-Page Build Blueprint

```
NEXORA
■
■■■ PHASE 0  → Freeze product + architecture
■
■■■ PHASE 1  → React + Express + MongoDB + CI
■
■■■ PHASE 2  → JWT auth + refresh sessions + protected APIs
■
■■■ PHASE 3  → Tasks + Study + Profile + Dashboard
■
■■■ PHASE 4  → Skills + Career + Projects + Analytics
■
■■■ PHASE 5  → AI adapter + LLM chat + conversation storage
■
■■■ PHASE 6  → Short/session/long-term memory + privacy controls
■
■■■ PHASE 7  → Skill gap + roadmap + recommendations + feedback
■
■■■ PHASE 8  → Vector memory + SSE (optional Phase 1.5)
■
■■■ PHASE 9  → Security + Jest + Supertest + RTL + Playwright
■
■■■ PHASE 10 → GitHub Actions + Vercel + Render/Railway + Atlas
■
■■■ PHASE 11 → UI/UX + accessibility + loading/error states
■
■■■ PHASE 12 → README + API docs + demo + resume/viva evidence

CORE RUNTIME PIPELINE

USER
 ↓
REACT
 ↓
REST / JSON
 ↓
NODE + EXPRESS
 ↓
AUTH / VALIDATION / RATE LIMIT / LOGGING
 ↓
CONTROLLER
 ↓
SERVICE
 ■■■■■■■■■■■■■■■■→ MONGODB
 ■
 ■■■■■■■■■■■■■■■■→ AI ENGINE
                    ↓
              MEMORY RETRIEVER
                    ↓
              CONTEXT BUILDER
                    ↓
              PROMPT MANAGER
                    ↓
            LLM PROVIDER ADAPTER
                    ↓
           HOSTED PRETRAINED LLM
                    ↓
              RESPONSE PARSER
                    ↓
               JSON / SSE
                    ↓
                 REACT
                    ↓
             STUDENT ACTION
                    ↓
                MONGODB
```

Final principle: NEXORA is not valuable because it contains an LLM. It is valuable because the application combines structured student data, deterministic analytics, career/skill knowledge, user-controlled memory, bounded context retrieval, feedback-aware recommendations, secure APIs and a polished student workflow around the LLM.

This document should be used as the implementation checklist while developing NEXORA. Freeze the architecture, build the core end-to-end, then add advanced AI retrieval/streaming only after the stable product exists.